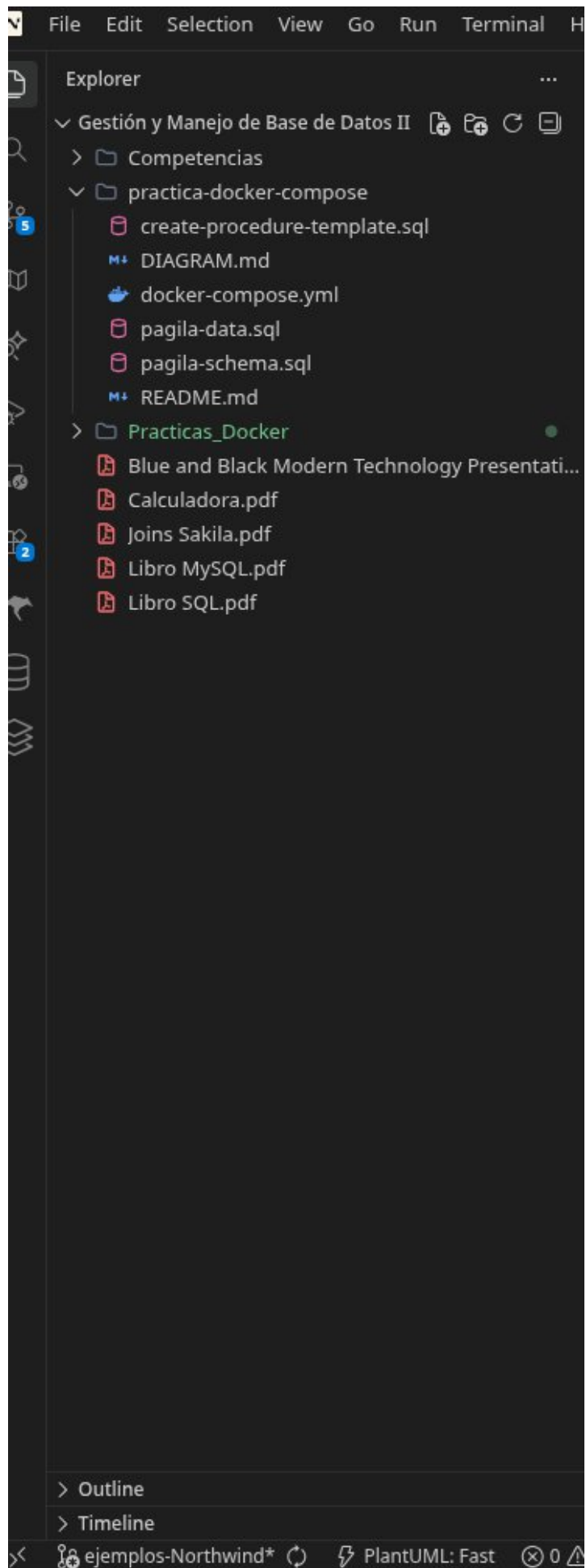
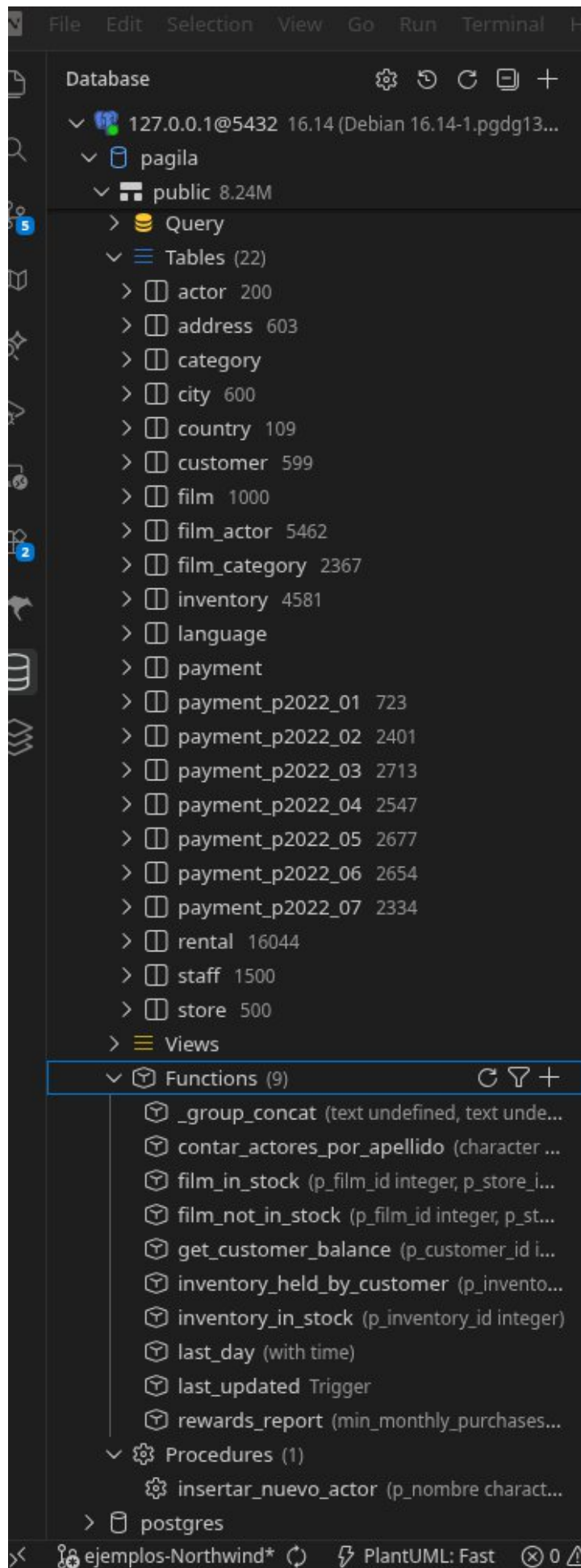






Database

127.0.0.1@5432 16.14 (Debian 16.14-1.pgdg13...)

pagila

public 8.24M

Query

Tables (22)

actor 200

address 603

category

city 600

country 109

customer 599

film 1000

film_actor 5462

film_category 2367

inventory 4581

language

payment

payment_p2022_01 723

payment_p2022_02 2401

payment_p2022_03 2713

payment_p2022_04 2547

payment_p2022_05 2677

payment_p2022_06 2654

payment_p2022_07 2334

rental 16044

staff 1500

store 500

Views

Functions (10)

_group_concat (text undefined, text unde...)

contar_actores_por_apellido (character ...)

film_in_stock (p_film_id integer, p_store_i...

film_not_in_stock (p_film_id integer, p_st...

get_customer_balance (p_customer_id i...

inventory_held_by_customer (p_invento...

inventory_in_stock (p_inventory_id integer)

last_day (with time)

last_updated Trigger

rewards_report (min_monthly_purchases...

Procedures (1)

insertar_nuevo_actor (p_nombre charact...

postgres

docker-compose.yml

query_funcion.sql

query_procedimiento.sql

home > raulito > .dbclient > query > 1779754265385@@127.0.0.1@5432@pagila@public > query_procedimiento.sql > ...

1

-- Insertar

D>Run

2

CALL insertar_nuevo_actor('RAUL', 'ROCHA');

3

-- Verificar

D>Run | +Tab | JSON

4

5

SELECT * FROM actor WHERE first_name = 'RAUL' AND last_name = 'ROCHA'; 2ms

actor

Result2

Result3

Search Results

Cost: 3ms < 1 > Total 1

actor_id Integer

first_name text

last_name text

last_update timestamp with time

201

RAUL

ROCHA

2026-05-26 22:18:57.6128.

Database

127.0.0.1@5432 16.14 (Debian 16.14-1.pgdg13...

pagila

public 8.24M

Query

Tables (22)

actor 200

address 603

category

city 600

country 109

customer 599

film 1000

film_actor 5462

film_category 2367

inventory 4581

language

payment

payment_p2022_01 723

payment_p2022_02 2401

payment_p2022_03 2713

payment_p2022_04 2547

payment_p2022_05 2677

payment_p2022_06 2654

payment_p2022_07 2334

rental 16044

staff 1500

store 500

Views

Functions (10)

_group_concat (text undefined, text unde...

contar_actores_por_apellido (character ...

film_in_stock (p_film_id integer, p_store_i...

film_not_in_stock (p_film_id integer, p_st...

get_customer_balance (p_customer_id i...

inventory_held_by_customer (p_invento...

inventory_in_stock (p_inventory_id integer)

last_day (with time)

last_updated Trigger

rewards_report (min_monthly_purchases...

Procedures (1)

insertar_nuevo_actor (p_nombre charact...

postgres

docker-compose.yml

query_funcion.sql X

query_procedimiento.sql

home > raulito > .dbclient > query > 1779754265385@@127.0.0.1@5432@pagila@public > query_funcion.sql > ...

6

7

8

9

10

11

12

13

14

15

16

17

18

19

-- A. Verificar que Penelope Guinness existe (con una sola N)

>Run | +Tab | JSON

SELECT first_name, last_name FROM actor WHERE last_name = 'GUINNESS'; 3ms

-- B. Probar la función con el apellido correcto

>Run | +Tab | JSON

SELECT contar_actores_por_apellido('GUINNESS') AS total_mayusculas;

-- C. Probar robustez con minúsculas

>Run | +Tab | JSON

SELECT contar_actores_por_apellido('guinness') AS total_minusculas;

-- D. Probar con un apellido que NO existe

>Run | +Tab | JSON

SELECT contar_actores_por_apellido('ZAPATA') AS total_inexistente;

actor X Result2 Result3

Search Results

* first_name text

* last_name text

> PENELOPE GUINNESS

> SEAN GUINNESS

> ED GUINNESS

Cost: 4ms < 1 > Total 3

